

2026 VLSI Design and Implementation

LAB2 Coding Style — Dart Game

Objection

In this lab, students will practice **using standard coding styles taught in the class** to implement a Verilog hardware module.

Due Date: 2026/5/1 12:00 p.m.

Lab Download

1. Download the provided environment using the following command.
tar xvf /home/TA00/VLSI_2026/Lab2.tar
2. The extracted LAB directory contains:
 - **00_TESTBED**: Contains the testbench and pattern files .
 - **01_RTL**: Your workspace for the Verilog design (**DART.v**).
 - **02_SYN**: Synthesis scripts and working directory.
 - **03_GATE**: Gate-level netlist and simulation files.

Design Description

In this lab, you are required to design a hardware controller for a two-player dart game. The game follows a modified version of the classic "301" rules.



Fig. 1. Dartboard

1. Initial State & Gameplay

- The game consists of two players: Player 0 and Player 1.
- Both players start the game with exactly **301 points**.
- Players take turns throwing darts, strictly alternating between Player 0 and Player 1. Each player will throw exactly **10 darts** per game (totaling 20 darts per game).

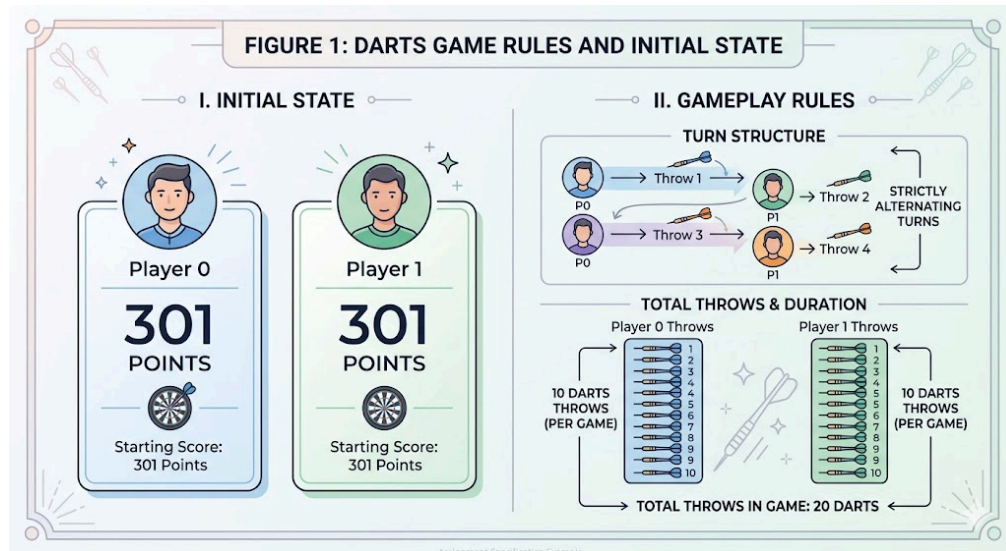


Fig. 2. Game Rules and Input Sequence

2. Score Calculation

- Each dart has a base score (**points**) ranging from 0 to 20, plus a special center bullseye represented by 25.
- If a dart hits the double ring (**is_double** is high), the base score is multiplied by 2.
- If a dart hits the triple ring (**is_triple** is high), the base score is multiplied by 3.
- The center bullseye (**points** = 25) is always treated as a double (**is_double** will must be high at the same time), yielding a total of 50 points.

3. Deducting Points & "Bust" Rule

- The total score of each valid dart throw is subtracted from the player's current score.
- **Bust:** If a dart's score would cause the player's remaining score to drop **below 0**, it is considered a "Bust." The score of that specific dart is ignored, and the player's total score remains unchanged for that throw.

4. Winning Conditions

- **Standard Win:** If both players finish their 10 throws without reaching exactly 0, the player with the lowest remaining score wins.
- **Early Win:** If a player's score drops to **exactly 0**, they win the game immediately. Any darts thrown by either player after someone has reached 0 are considered invalid and must not affect the scores. **However, the remaining darts will still be given.**

⇒ **Input will always be 20 cycles**

- **Tie-breaker:** If both players have the exact same remaining score at the end of the game (at final round), **Player 0** is declared the winner.

5. Output Specifications

- The system will sequentially receive 20 valid inputs (`in_valid = 1`) per game.
- After the 20th valid dart has been processed, the module must pull `out_valid` high for exactly **1 clock cycle**. Simultaneously, `out_winner` must output the ID of the winning player (0 or 1).

Game Exapmle

- Example 1 — Normal Condition

Round	Player 0 Throw		Player 1 Throw		Initial: 301	Player 0 Score	Player 1 Score
1	[cycle 1]	25*2	[cycle 2]	11		251	290
2	[cycle 3]	16	[cycle 4]	10		235	280
3	[cycle 5]	18*3	[cycle 6]	10		181	270
4	[cycle 7]	6	[cycle 8]	10		175	260
5	[cycle 9]	18*2	[cycle 10]	10		139	250
6	[cycle 11]	16	[cycle 12]	10		123	240
7	[cycle 13]	12*3	[cycle 14]	10		87	230
8	[cycle 15]	15	[cycle 16]	10		72	220
9	[cycle 17]	20*3	[cycle 18]	10		12	210
10	[cycle 19]	20*3	[cycle 20]	10		12	200

Bust !!!
60 > 12, so the score of the dart is ignored.

12 < 200, P0 Win

Fig. 3. Standard win (Bust Rule)

- Example 2 — Early Win



Fig. 4. Early Win (A player reach 0 score)

- Example 3 — Tie



Fig. 5. Tie (Same points after the final round)

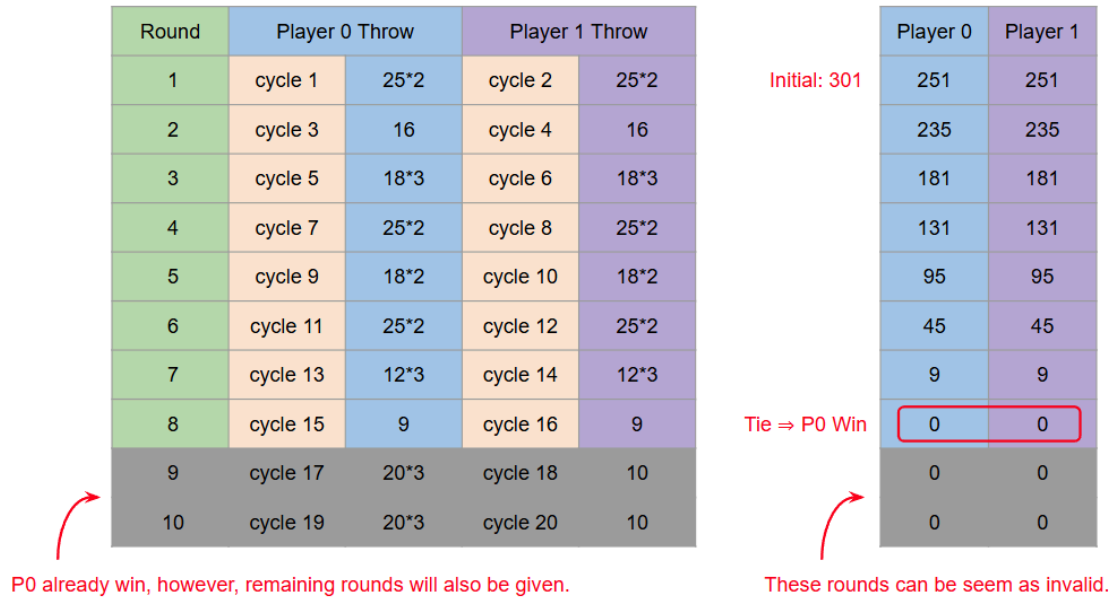


Fig. 6. Tie (Two player reach 0 score at the same time)

Signal Description

Signal	In / Out	Bit	Description
clk	In	1	System Clock.
rst_n	In	1	Asynchronous active-low reset.
in_valid	In	1	High when the inputs are valid.
player	In	1	Indicates the current thrower (0 for P0, 1 for P1).
is_double	In	1	High when the dart hits the double ring ($points * 2$).
is_triple	In	1	High when the dart hits the triple ring ($points * 3$).
points	In	5	The base score of the dart (0~20, and 25).
out_valid	Out	1	High when the output <code>out_winner</code> is valid.
out_winner	Out	1	Indicates the game winner (0 for P0, 1 for P1). Valid only when <code>out_valid</code> is high.

Table. 1. I/O port list

Design Constraints

Coding Style

1. Create a separate Verilog file for each module, named <module_name>.v.
2. Use consistent ordering for bit vectors [high:low]
3. The top module can only instantiate submodules.
4. Do not mix module instantiations and procedural code in a single module.
5. Each module may only contain one always block, which must drive a single set of signals.
(Exception: Finite State Machines (FSMs) are permitted to use two always blocks, one for state and another for next state.)
6. Each module may contain multiple assign statements.
7. Continuous assignments (assign statements) and procedural blocks (always blocks) must not coexist within a single module.
8. All conditional logic must be implemented using case statements. The use of if-else statements is prohibited, except when handling the reset (rst_n) signal.

Top module: DART.v

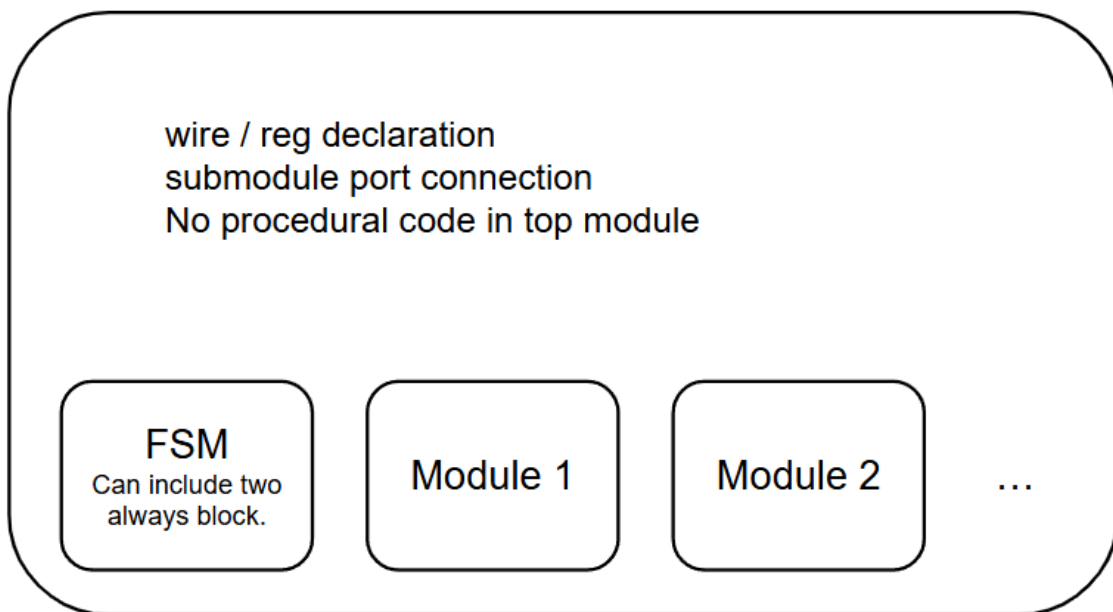


Fig. 7. Module hierarchy

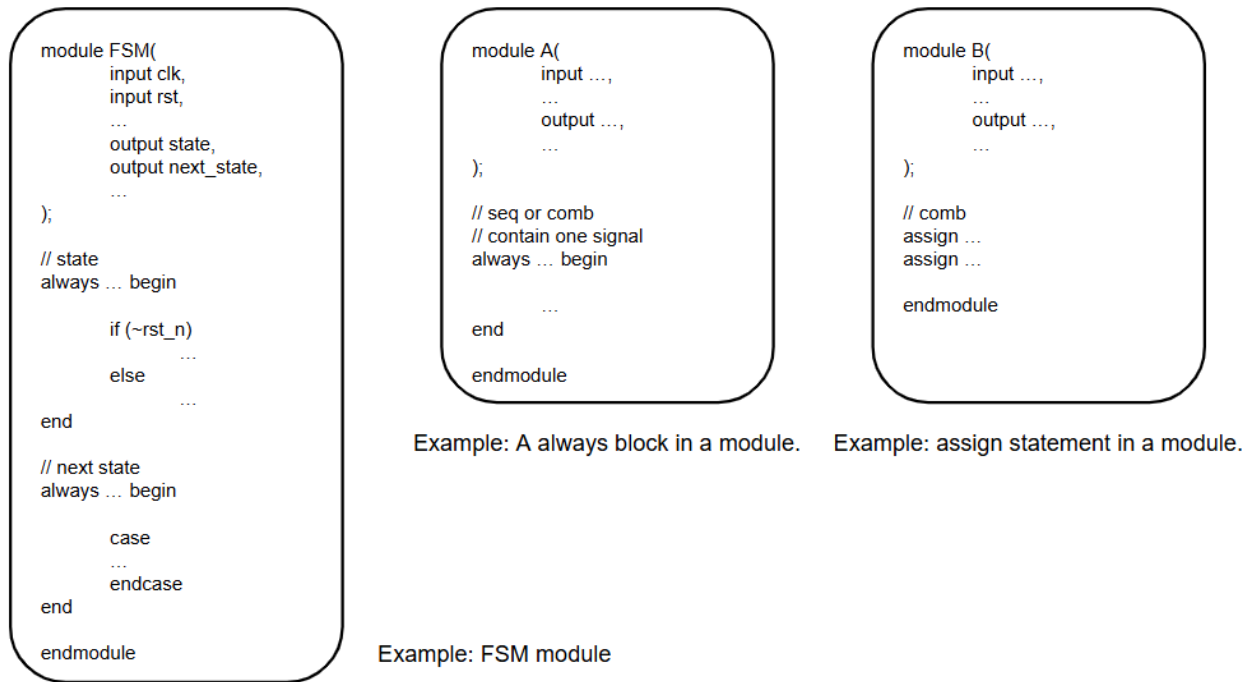


Fig. 8. Example of submodules

01_RTL

1. Top module name: **DART.v**
2. **RTL code must conform to the coding style rules mentioned above.**
3. The reset signal (**rst_n**) would be given only once at the beginning of simulation. All output signals should be reset after the reset signal is asserted.
4. The **out_winner** should be zero after your **out_valid** is pulled down.
5. The **out_valid** signal cannot overlap with the **in_valid** signal at any time.
6. The **out_valid** signal should be high for exactly **1 clock cycle**.
7. The next inputs will come in the next 3 ~ 7 negative edges of the clock after your **out_valid** is pulled down.
8. **Your design must pass all hidden testcase.**

02_SYN

1. The synthesis result cannot contain any **Latch** and any **Error**.
2. The input / output delay is set to $0.5 * (\text{clock period})$, and the output loading is set to 0.05.
3. You can adjust your clock period by yourself. The precision of the clock period is 0.1, for example, 45.0 is allowed, 44.55 is not allowed.
4. After synthesis, the design must meet timing requirements. The area report is valid only if the final slack reported in **DART.timing** is **MET**.

```

Startpoint: in_valid (input port clocked by clk)
Endpoint: mat_a_reg_1_0_0_0_0
          (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max

```

Point	Incr	Path
clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
input external delay	10.00	10.00 f
in_valid (in)	0.00	10.00 f
U1789/Y (INVX1)	0.26	10.26 r
U1795/Y (NOR2XL)	0.12	10.38 f
U3208/Y (NAND3BX1)	0.15	10.53 r
U1791/Y (INVXL)	0.11	10.64 f
U3339/Y (NAND3XL)	0.14	10.78 r
U3602/Y (OR2X1)	0.80	11.58 r
U1799/Y (CLKINVX3)	0.62	12.20 f
U3659/Y (A022X1)	0.38	12.59 f
mat_a_reg_1_0_0_0_0/D (DFFQX1)	0.00	12.59 f
data arrival time		12.59
clock clk (rise edge)	20.00	20.00
clock network delay (ideal)	0.00	20.00
clock uncertainty	-0.10	19.90
mat_a_reg_1_0_0_0_0/CK (DFFQX1)	0.00	19.90 r
library setup time	-0.23	19.67
data required time		19.67
data required time		19.67
data arrival time		-12.59
slack (MET)		7.08

Fig. 9. Timing report with no timing violation

5. You can use `08_check` to check your synthesis result.

```

[chfan@localhost 02_SYN]$ ./08_check
=====
--> V Latch Checked!
--> V Width Mismatch Checked!
--> V No Error in syn.log!
--> V Timing (MET) Checked!
=====
--> V 02_SYN Success !!
=====
Cycle: 20.00
Area: 63126.305690
Gate count: 6326
Dynamic: Total Dynamic Power    = 923.2034 uW  (100%)
Leakage: Cell Leakage Power     = 53.2685 uW

[chfan@localhost 02_SYN]$ ./08_check
=====
--> X There is Latch in this design ---
--> V Width Mismatch Checked!
--> V No Error in syn.log!
--> V Timing (MET) Checked!
=====
--> X 02_SYN Fail !! Please check out syn.log file.
=====
Cycle: 20.00
Area: 63126.305690
Gate count: 6326
Dynamic: Total Dynamic Power    = 923.2034 uW  (100%)
Leakage: Cell Leakage Power     = 53.2685 uW

```

Fig. 10. `08_check` example

03_GATE

1. All requirements in 01_RTL must be met after including timing checks in Gate-Level simulation.

Waveform Examples

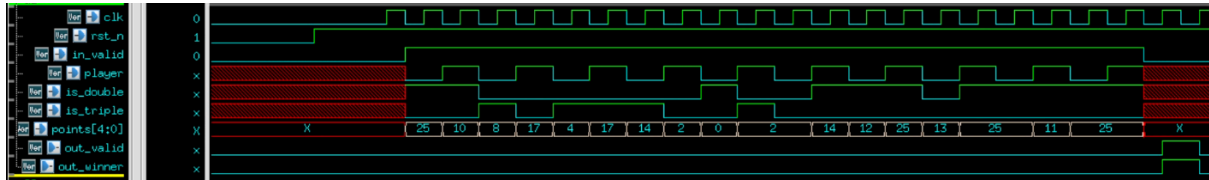


Fig. 11. Input sequence and output

Grading Policy

1. Function Validity: 50%

Passing RTL, SYN and Gate-level simulation without timing violations.

2. Report & Coding Style: 30%

This 30% grade component evaluates both the correctness of your coding style implementation and the completeness of the report.

The report must include:

- Your complete Verilog code (either as screenshots or plain text).
- Explain how each module adheres to the specified coding style rules in brief.

3. Performance: 20%

Performance Score = Latency * Clock Cycle Time * Area

※The Latency is the clock cycles between the falling edge of the in_valid and the rising edge of the first out_valid.

※Clock Cycle Time is defined by yourself.

※Area means “Total cell area” in your area report.

4. Due date: 2026/5/1 12:00 p.m.

Please submit the following zip files to the E3 system before 12:00 at noon on May 1

- vlsixx.zip (ex. vlsi00.zip)
 - vlsixx (folder)
 - vlsixx_report.pdf
 - {your cycle time}.txt (ex. 10.6.txt)
 - DART.v
 - <Your_module>.v (Can more than one module)

5. If uploaded files violate the naming rule, you will get 5 deduct points.

Notes

- Submission
 - 1st Demo
Submit and pass all test cases before 12:00 noon on May 1, 2026. The score will be calculated according to the grading policy.
 - 2nd Demo
If you do not pass all test cases or fail to submit (in the 1st Demo), and submit and pass all test cases before 12:00 noon on May 8, 2026, you will receive the Function score(50%) and Report score (30%).
- Reference Commands
 - 01_RTL/ (RTL simulation) `./01_run_vcs_rtl`
 - 02_SYN/ (Synthesis) `./01_run_dc_shell`
 - 03_GATE / (Gate-level simulation) `./01_run_vcs_gate`
- How to include submodule in `DART.v`
In the top of `DART.v`, include all your submodules here.

```
1 // =====  
2 // include your module here  
3 // e.g. `include "module1.v"  
4 // =====  
5 // `include "<your_module>.v"
```

Fig. 12. 01_RTL/DART.v

- How to change cycle time
You can adjust cycle time in `00_TESTBED/PATTERN.v` and `02_SYN/syn.tcl`.
※Note: Both cycle time configurations must be the same.

```
00_TESTBED >  PATTERN.v  
1  `ifdef RTL  
2 |   `define CYCLE_TIME 20.0  
3 | `endif  
4  `ifdef GATE  
5 |   `define CYCLE_TIME 20.0  
6 | `endif
```

Fig. 13. 00_TESTBED/PATTERN.v

```
02_SYN > ⚙ syn.tcl
1  #=====
2  #
3  # Synopsys Synthesis Scripts (Design Vision dcl mode)
4  #
5  #=====
6
7  #=====
8  # (A) Global Parameters
9  #=====
10 set DESIGN "MATRIX_MULT"
11 set CYCLE 20
12 set INPUT_DLY [expr 0.5*$CYCLE]
13 set OUTPUT_DLY [expr 0.5*$CYCLE]
```

Fig. 14. 02_SYN/syn.tcl